

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0515

Slot

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4) Assessment Tool Weightage: 40%

QUESTIONS:3

Total Marks 30

Duration :60 Minutes

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	
---------------	----	---	--	--	----------------------------------	--

Annexure A
Experiments

Code of Conduct:

I K.Mounith(19242509) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Mounith

Rubrics for Evaluation

1. Harris Corner Detection Analysis

Perform Harris corner detection on images containing varying textures and patterns. Explain the concept of corner detection and its importance in computer vision tasks. Use appropriate parameters and tools to execute the experiment, document the detected corner points clearly, and analyze the distribution, accuracy, and significance of the detected corners.

A.import cv2

import numpy as np

import matplotlib.pyplot as plt

from tkinter import Tk, filedialog

Harris Corner Detection

Hide the main Tkinter window

root = Tk()

root.withdraw()

```
# Select an image

file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[
        ("Image Files", "*.jpg *.jpeg *.png *.bmp")
    ]
)

# Check whether an image was selected

if not file_path:
    print("No image selected.")
    exit()

# Read image

image = cv2.imread(file_path)

if image is None:
    print("Error: Could not load the selected image.")
    exit()

# Convert BGR to RGB

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert to grayscale

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
# Convert image to float32
```

```
gray_float = np.float32(gray)
```

```
# Harris Corner Detection
```

```
dst = cv2.cornerHarris(
```

```
    gray_float,
```

```
    blockSize=2,
```

```
    ksize=3,
```

```
    k=0.04
```

```
)
```

```
# Dilate corner points
```

```
dst = cv2.dilate(dst, None)
```

```
# Set threshold
```

```
threshold = 0.01 * dst.max()
```

```
# Copy original image
```

```
result = image_rgb.copy()
```

```
# Mark corners in red
```

```
result[dst > threshold] = [255, 0, 0]
```

```
# Count corners
```

```
corner_count = np.sum(dst > threshold)
```

```
# Display information
```

```
print("-----")
```

```
print("HARRIS CORNER DETECTION ANALYSIS")
```

```
print("-----")
```

```
print("Selected Image:", file_path)
```

```
print("Number of detected corners:", corner_count)
```

```
print("-----")
```

```
# Display images
```

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.imshow(image_rgb)
```

```
plt.title("Original Image")
```

```
plt.axis("off")
```

```
plt.subplot(1, 2, 2)
```

```
plt.imshow(result)
```

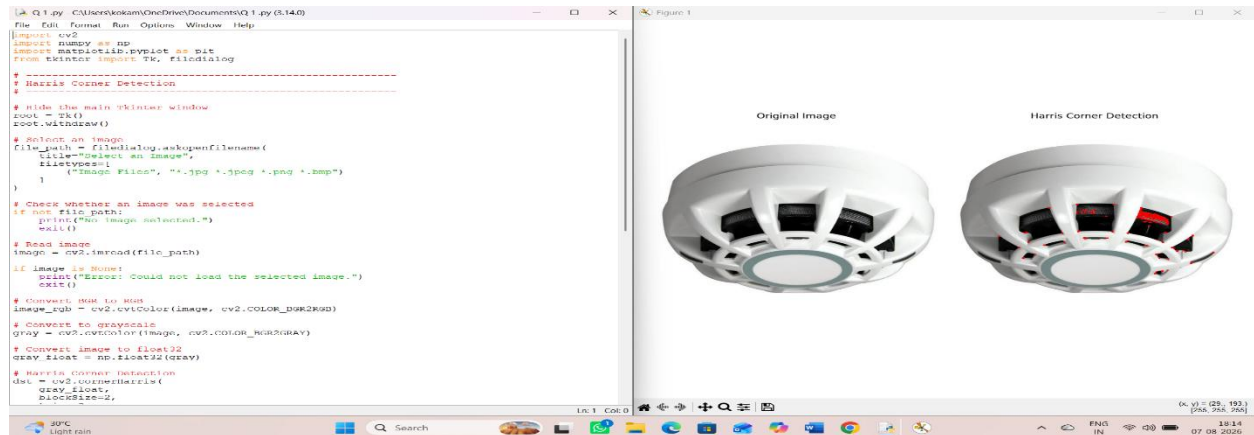
```
plt.title("Harris Corner Detection")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
OUTPUT;
```



2. Effect of Noise on Corner Detection

Design an experiment to study the effect of noise on Harris corner detection performance. Explain the purpose of introducing noise and its influence on image features. Add noise systematically, perform corner detection using suitable tools, document observations for original and noisy images, and analyze the impact of noise along with possible improvement techniques.

A.import cv2

import numpy as np

import matplotlib.pyplot as plt

from tkinter import Tk, filedialog

EXPERIMENT 2

Effect of Noise on Harris Corner Detection

Open file selection window

root = Tk()

root.withdraw()

file_path = filedialog.askopenfilename(

title="Select an Image",

```
filetypes=[  
    ("Image Files", "*.jpg *.jpeg *.png *.bmp")  
]  
)
```

```
# Check image selection
```

```
if not file_path:
```

```
    print("No image selected.")
```

```
    exit()
```

```
# Read image
```

```
image = cv2.imread(file_path)
```

```
if image is None:
```

```
    print("Error: Image could not be loaded.")
```

```
    exit()
```

```
# Convert BGR to RGB
```

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
# Convert image to grayscale
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
# -----
```

```
# Function to Add Gaussian Noise
```

```
# -----
```

```
def add_gaussian_noise(image, noise_level):
```

```
    # Generate Gaussian noise
```

```
    noise = np.random.normal(
```

```
        0,
```

```
        noise_level,
```

```
        image.shape
```

```
    )
```

```
    # Add noise
```

```
    noisy_image = image.astype(np.float32) + noise
```

```
    # Keep pixel values between 0 and 255
```

```
    noisy_image = np.clip(noisy_image, 0, 255)
```

```
    return noisy_image.astype(np.uint8)
```

```
# -----
```

```
# Function for Harris Corner Detection
```

```
# -----
```

```
def detect_corners(image):
```



```
# Convert to grayscale  
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
```

```
# Convert to float32  
gray = np.float32(gray)
```

```
# Harris Corner Detection  
corners = cv2.cornerHarris(  
    gray,  
    blockSize=2,  
    ksize=3,  
    k=0.04  
)
```

```
# Dilate corner response  
corners = cv2.dilate(corners, None)
```

```
# Threshold  
threshold = 0.01 * corners.max()
```

```
# Copy image  
result = image.copy()
```

```
# Mark detected corners in red  
result[corners > threshold] = [255, 0, 0]
```

```
# Count corners
```

```
count = np.sum(corners > threshold)
```

```
return result, count
```

```
# -----
```

```
# Generate Noisy Images
```

```
# -----
```

```
np.random.seed(42)
```

```
noisy_low = add_gaussian_noise(image_rgb, 10)
```

```
noisy_medium = add_gaussian_noise(image_rgb, 25)
```

```
noisy_high = add_gaussian_noise(image_rgb, 50)
```

```
# -----
```

```
# Harris Corner Detection
```

```
# -----
```

```
result_original, count_original = detect_corners(image_rgb)
```

```
result_low, count_low = detect_corners(noisy_low)
```

```
result_medium, count_medium = detect_corners(noisy_medium)
```

```
result_high, count_high = detect_corners(noisy_high)
```

```
# -----
```

```
# Print Results
```

```
# -----
```

```
print("\n=====")
```

```
print(" EFFECT OF NOISE ON HARRIS CORNER DETECTION")
```

```
print("=====")
```

```
print("Image:", file_path)
```

```
print("\nDetected Corner Points:")
```

```
print("-----")
```

```
print("Original Image :", count_original)
```

```
print("Low Noise   :", count_low)
```

```
print("Medium Noise :", count_medium)
```

```
print("High Noise   :", count_high)
```

```
print("-----")
```

```
# -----
```

```
# Display Results
```

```
# -----
```

```
plt.figure(figsize=(15, 10))
```

```
# Original
```

```
plt.subplot(2, 4, 1)
```

```
plt.imshow(image_rgb)
```

```
plt.title("Original Image")
```

```
plt.axis("off")
```

```
plt.subplot(2, 4, 5)
```

```
plt.imshow(result_original)
```

```
plt.title("Original - Corners")
```

```
plt.axis("off")
```

```
# Low Noise
```

```
plt.subplot(2, 4, 2)
```

```
plt.imshow(noisy_low)
```

```
plt.title("Low Noise")
```

```
plt.axis("off")
```

```
plt.subplot(2, 4, 6)
```

```
plt.imshow(result_low)
```

```
plt.title("Low Noise - Corners")
```

```
plt.axis("off")
```

```
# Medium Noise
```

```
plt.subplot(2, 4, 3)
```

```
plt.imshow(noisy_medium)
```

```
plt.title("Medium Noise")
```

```
plt.axis("off")
```

```
plt.subplot(2, 4, 7)
```

```
plt.imshow(result_medium)
```

```
plt.title("Medium Noise - Corners")
```

```
plt.axis("off")
```

```
# High Noise
```

```
plt.subplot(2, 4, 4)
```

```
plt.imshow(noisy_high)
```

```
plt.title("High Noise")
```

```
plt.axis("off")
```

```
plt.subplot(2, 4, 8)
```

```
plt.imshow(result_high)
```

```
plt.title("High Noise - Corners")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----  
# Noise Reduction Using Gaussian Blur  
# -----  
  
# Apply Gaussian filtering to high-noise image  
filtered_image = cv2.GaussianBlur(  
    noisy_high,  
    (5, 5),  
    0  
)  
  
# Detect corners after noise reduction  
filtered_result, filtered_count = detect_corners(  
    filtered_image  
)  
  
print("\nAfter Gaussian Noise Reduction:")  
print("Corners detected:", filtered_count)  
  
# -----  
# Display Noise Reduction Result  
# -----
```

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.imshow(noisy_high)
```

```
plt.title("High Noise Image")
```

```
plt.axis("off")
```

```
plt.subplot(1, 2, 2)
```

```
plt.imshow(filtered_result)
```

```
plt.title("After Gaussian Filtering - Corners")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----
```

```
# Graph: Noise Level vs Number of Corners
```

```
# -----
```

```
noise_levels = [0, 10, 25, 50]
```

```
corner_counts = [
```

```
    count_original,
```

```
    count_low,
```

```
    count_medium,
```

```

        count_high
    ]

plt.figure(figsize=(8, 5))

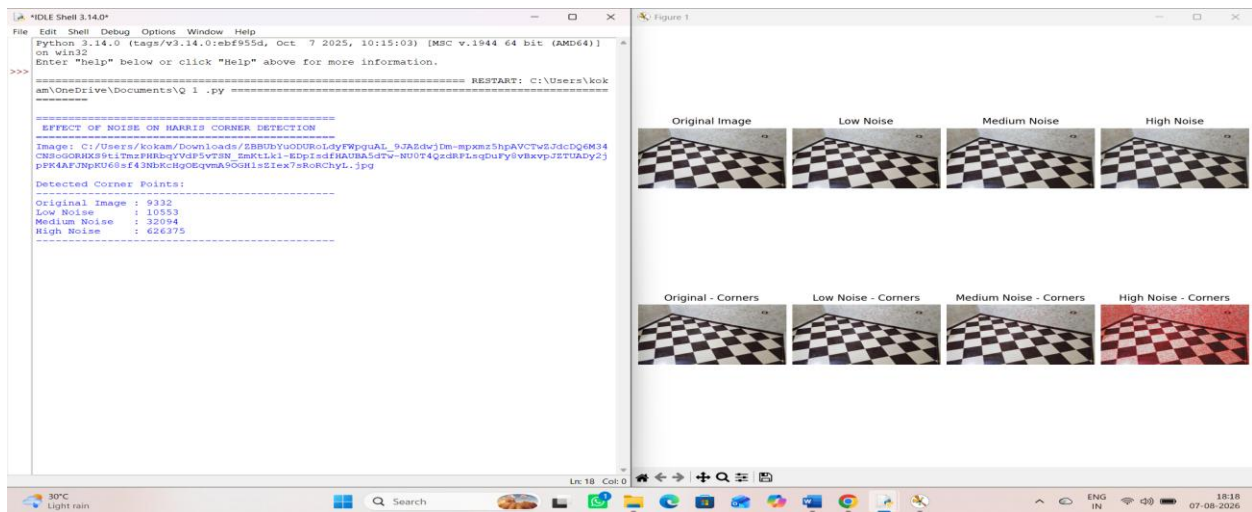
plt.plot(
    noise_levels,
    corner_counts,
    marker="o"
)

plt.xlabel("Noise Level")
plt.ylabel("Number of Detected Corners")
plt.title("Effect of Noise on Harris Corner Detection")

plt.grid(True)
plt.show()

```

OUTPUT;



3. Hough Transform for Shape Detection

Conduct an experiment using the Hough Transform to detect lines or circles in an image. Explain the principles and significance of the Hough Transform in shape detection. Execute the detection process using appropriate tools, record the detected shapes and their parameters, and analyze the detection accuracy under different image conditions.

```
A.import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from tkinter import Tk, filedialog
```

```
# -----
```

```
# EXPERIMENT 3
```

```
# HOUGH TRANSFORM FOR SHAPE DETECTION
```

```
# -----
```

```
# Open file selection window
```

```
root = Tk()
```

```
root.withdraw()
```

```
file_path = filedialog.askopenfilename(
```

```
    title="Select an Image",
```

```
    filetypes=[
```

```
        ("Image Files", "*.jpg *.jpeg *.png *.bmp")
```

```
    ]
```

```
)
```

```
# Check image selection
```

```
if not file_path:
```

```
print("No image selected.")
```

```
exit()
```

```
# Read image
```

```
image = cv2.imread(file_path)
```

```
if image is None:
```

```
    print("Error: Could not load image.")
```

```
    exit()
```

```
# Convert BGR to RGB
```

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
# Convert to grayscale
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
# Smooth image to reduce noise
```

```
gray_blur = cv2.GaussianBlur(gray, (5, 5), 0)
```

```
# =====
```

```
# 1. HOUGH LINE TRANSFORM
```

```
# =====
```

```
# Detect edges using Canny
```

```
edges = cv2.Canny(
```

```
gray_blur,  
50,  
150,  
apertureSize=3  
)
```

```
# Detect lines
```

```
lines = cv2.HoughLinesP(  
    edges,  
    rho=1,  
    theta=np.pi / 180,  
    threshold=80,  
    minLineLength=50,  
    maxLineGap=10  
)
```

```
# Copy image for drawing lines
```

```
line_result = image_rgb.copy()
```

```
line_count = 0
```

```
if lines is not None:
```

```
    for line in lines:
```

```
        x1, y1, x2, y2 = line[0]
```

```
# Draw detected line
```

```
cv2.line(  
    line_result,  
    (x1, y1),  
    (x2, y2),  
    (255, 0, 0),  
    3  
)
```

```
line_count += 1
```

```
# =====
```

```
# 2. HOUGH CIRCLE TRANSFORM
```

```
# =====
```

```
circles = cv2.HoughCircles(  
    gray_blur,  
    cv2.HOUGH_GRADIENT,  
    dp=1.2,  
    minDist=30,  
    param1=100,  
    param2=30,  
    minRadius=10,  
    maxRadius=200
```

```
)
```

```
# Copy image for drawing circles
```

```
circle_result = image_rgb.copy()
```

```
circle_count = 0
```

```
if circles is not None:
```

```
    circles = np.uint16(  
        np.around(circles)  
    )
```

```
    for circle in circles[0, :]:
```

```
        x, y, r = circle
```

```
        # Draw circle
```

```
        cv2.circle(  
            circle_result,  
            (x, y),  
            r,  
            (0, 255, 0),  
            3  
        )
```

```
# Draw center
```

```
cv2.circle(  
    circle_result,  
    (x, y),  
    3,  
    (255, 0, 0),  
    -1  
)
```

```
circle_count += 1
```

```
# =====
```

```
# DISPLAY RESULTS
```

```
# =====
```

```
print("\n=====")
```

```
print(" HOUGH TRANSFORM FOR SHAPE DETECTION")
```

```
print("=====")
```

```
print("\nImage:", file_path)
```

```
print("\nDetected Shapes")
```

```
print("-----")
```

```
print("Number of Lines :", line_count)
```

```
print("Number of Circles :", circle_count)
```

```
print("-----")
```

```
# Display results
```

```
plt.figure(figsize=(15, 10))
```

```
# Original image
```

```
plt.subplot(2, 2, 1)
```

```
plt.imshow(image_rgb)
```

```
plt.title("Original Image")
```

```
plt.axis("off")
```

```
# Edge image
```

```
plt.subplot(2, 2, 2)
```

```
plt.imshow(edges, cmap="gray")
```

```
plt.title("Canny Edge Detection")
```

```
plt.axis("off")
```

```
# Lines
```

```
plt.subplot(2, 2, 3)
```

```
plt.imshow(line_result)
```

```
plt.title("Hough Line Detection")
```

```
plt.axis("off")
```

```
# Circles
```

```
plt.subplot(2, 2, 4)
```

```
plt.imshow(circle_result)
```

```
plt.title("Hough Circle Detection")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print("\nLINE PARAMETERS")
```

```
print("-----")
```

```
if lines is not None:
```

```
    for i, line in enumerate(lines):
```

```
        x1, y1, x2, y2 = line[0]
```

```
        # Calculate length
```

```
        length = np.sqrt(
```

```
            (x2 - x1) ** 2 +
```

```
            (y2 - y1) ** 2
```

```
        )
```

```
        # Calculate angle
```

```
        angle = np.degrees(
```

```
            np.arctan2(
```

```
                y2 - y1,
```

```
                x2 - x1
```

```
            )
```

```
        )
```

```
        print(
```

```
            f"Line {i + 1}: "
```

```
            f"({x1},{y1}) -> ({x2},{y2}), "
```

```
            f"Length = {length:.2f}, "
```

```
            f"Angle = {angle:.2f} degrees"
```

```
        )
```


else:

```
print("No lines detected.")
```

```
# =====
```

```
# PRINT CIRCLE PARAMETERS
```

```
# =====
```

```
print("\nCIRCLE PARAMETERS")
```

```
print("-----")
```

if circles is not None:

```
for i, circle in enumerate(circles[0, :]):
```

```
    x, y, r = circle
```

```
    print(
```

```
        f"Circle {i + 1}: "
```

```
        f"Center = ({x},{y}), "
```

```
        f"Radius = {r}"
```

```
    )
```

else:

```
print("No circles detected.")
```

```
print("\n=====")
```

```
print("Experiment completed successfully.")
```

```
print("=====")
```

OUTPUT;

